

Deep Learning Architectures Benchmark & OOP Framework

Research Internship Technical Report — Week 2 Submission | Sardar Ahmed

Author: Sardar Ahmed

Track: AI & Deep Learning Research

Datasets: MNIST (Vision), IMDB (NLP)

Institution: Alphasat Technologies

Architectures: ANN, CNN, RNN, LSTM

Framework: TensorFlow / Keras 3.x (OOP)

1. Executive Summary & OOP Architecture Transformation

This project establishes a modular, production-ready, Object-Oriented Deep Learning benchmark suite. The codebase refactors raw procedural neural network implementations into decoupled, single-responsibility classes: **ArtificialNeuralNetwork** (Step 1), **ConvolutionalNeuralNetwork** (Step 2), **RecurrentNeuralNetwork** (Step 3), **LSTMNeuralNetwork** (Step 4), and **DLModelComparator** (Step 5). A master orchestrator (**DeepLearningPipeline** in Main.py) coordinates training, serialization, metric computation, and automated visual artifact generation.

2. Computer Vision Domain: ANN vs. CNN (MNIST Benchmark)

The computer vision benchmark evaluates multi-class classification on 70,000 28×28 grayscale images across 10 digit classes.

- Artificial Neural Network (ANN / MLP):** Employs an input flattening stage followed by Dense(128, ReLU) and Dense(64, ReLU) hidden layers with Softmax output. While effective (**97.30% test accuracy**), ANNs lack spatial invariance and treat 2D spatial pixels as flat 1D vectors, requiring 109,386 parameters.
- Convolutional Neural Network (CNN):** Incorporates spatial inductive bias via dual Conv2D (32 and 64 filters of 3×3 kernel) and MaxPooling2D (2×2) layers, followed by Dropout (0.5) regularization. CNN preserves 2D topological locality, achieving a state-of-the-art **99.22% test accuracy** and a dramatic loss reduction to 0.0272.

3. Natural Language Domain: SimpleRNN vs. LSTM (IMDB Benchmark)

The sequential NLP benchmark evaluates binary sentiment classification on 50,000 polar movie reviews with a 10,000-word vocabulary and sequence truncation/padding to 200 tokens.

- Simple Recurrent Neural Network (SimpleRNN):** Utilizes standard recurrent hidden state transitions $h_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t + b_h)$. While lightweight (326,273 params), it suffers from exponential gradient decay (vanishing gradients), constraining long-term dependency retention (**72.16% test accuracy**).
- Long Short-Term Memory (LSTM):** Incorporates an explicit constant error carousel (CEC) governed by triple multiplicative gates: Forget Gate (f_t), Input Gate (i_t), and Output Gate (o_t). This cell memory architecture mitigates vanishing gradients across long token sequences, yielding a substantially higher **84.85% test accuracy**.

4. Quantitative Architectural Comparison & Metrics

Model Architecture	Domain & Dataset	Trainable Params	Test Accuracy	Test Loss
Artificial Neural Network (ANN)	Vision (MNIST)	109,386	97.30%	0.1149
Convolutional Neural Network (CNN)	Vision (MNIST)	225,034	99.22%	0.0272
Recurrent Neural Network (RNN)	NLP (IMDB)	326,273	72.16%	0.5605
Long Short-Term Memory (LSTM)	NLP (IMDB)	673,089	84.85%	0.4169

5. Architectural Takeaways & Research Insights

- Spatial Locality & Parameter Sharing:** CNNs reduce spatial redundancy compared to dense MLPs by learning translation-invariant spatial features through weight sharing and pooling, resulting in superior accuracy (99.22% vs 97.30%).

2. **Gradient Propagation in Sequential Data:** Standard RNNs struggle with vanishing gradients over sequence lengths $T > 50$, whereas LSTM's additive cell state update $\mathcal{C}_t = f_t \odot \mathcal{C}_{t-1} + i_t \odot \tilde{\mathcal{C}}_t$ preserves gradients effectively (+12.69% accuracy margin over SimpleRNN).

3. **Modularity & Production Readiness:** Encapsulating model lifecycles inside dedicated OOP classes allows straightforward hyperparameter tuning, standardized checkpointing (Models/*.keras), and automated multi-model pipeline execution via Main.py.